

Università degli Studi di Milano-Bicocca

Corso di Architetture Dati

ROBUSTEZZA DEI MODELLI DI MACHINE LEARNING ALL'AVVELENAMENTO DEL DATASET

*Analisi Sistematica dei Metodi di Corruzione sul Wisconsin Breast
Cancer Dataset*

Autori

Simone Rancati – Matricola 900052

Michele Piovanelli – Matricola 894433

Lorenzo Pina – Matricola 894396

Anno Accademico 2025/2026

Sommario

Abstract.....	3
1. Introduzione e Contesto del Progetto	4
1.1 Motivazione: Perché Studiare il Data Poisoning	4
1.2 Obiettivi della Ricerca Estesa.....	4
2. Dataset e Preprocessing di Base	5
2.1 Wisconsin Breast Cancer Dataset: Riepilogo.....	5
2.2 Preprocessing Iniziale e Split Train/Test.....	7
2.3 Architettura della Pipeline Sperimentale	7
3. Metodi di Avvelenamento del Dataset	9
3.1 Label Flipping (Avvelenamento dell'Etichetta).....	9
3.2 Gaussian Noise (Rumore Gaussiano sulle Feature).....	9
3.3 Outlier Injection (Iniezione di Valori Anomali).....	10
3.4 Missing Features (Valori Mancanti).....	10
3.5 Data Drift (Deriva dei dati).....	10
3.6 Random Feature Drop (Rimozione casuale di features).....	11
3.7 Top-K Drop (Rimozione di Features per importanza)	11
3.8 Duplicates (Duplicazione di osservazioni)	12
4. Architettura dei Modelli e Configurazione Sperimentale	13
4.1 Support Vector Machine (SVM).....	13
4.2 Random Forest.....	13
4.3 MLP (Multi-Layer Perceptron).....	13
4.4 Configurazione dell'Esperimento e Metriche	13
5. Risultati Sperimentali	15
5.1 Performance Baseline (Senza Corruzione)	15
5.2 Analisi di resistenza per Best e Worst 3.....	15
5.3 Impatto del Label Flipping	16
5.4 Impatto del Gaussian Noise.....	17
5.5 Impatto della Outlier Injection.....	18
5.6 Impatto dei Valori Mancanti	18
5.7 Impatto del Data Drift.....	19
5.8 Impatto del Random Feature Drop.....	19
5.9 Impatto del Top-K Drop.....	20
5.10 Impatto dei Duplicati	21
6. Analisi Comparativa dei Risultati.....	22
6.1 Gerarchia di Criticità dei Metodi di Corruzione.....	22

6.2 Confronto tra Modelli: Robustezza Complessiva	22
6.3 Effetto della Modalità di Selezione delle Feature sulla Robustezza.....	23
6.4 Analisi della Stabilità Statistica (20 Run)	23
7. Discussione e Implicazioni	24
7.1 Relazione rispetto alla baseline	24
7.2 Implicazioni per il Deployment Clinico.....	24
7.3 Limitazioni dello Studio.....	24
8. Conclusioni e prospettive future	26
8.1 Conclusioni	26
8.2 Prospettive Future.....	26

Abstract

Il presente studio analizza sistematicamente la robustezza dei modelli di Machine Learning rispetto a tecniche di data poisoning, utilizzando come benchmark il Wisconsin Breast Cancer Dataset. La ricerca esplora la resilienza di tre architetture — Support Vector Machine (SVM), Random Forest e Multi-Layer Perceptron (MLP) — attraverso un framework di corruzione controllata del training set.

La metodologia prevede l'implementazione di otto protocolli di corruzione: Label Flipping, iniezione di rumore gaussiano, inserimento di outlier, gestione di valori mancanti, Data Drift, rimozione casuale di feature, Top-K Drop basato sull'importanza delle variabili e duplicazione delle osservazioni. Tali alterazioni sono state applicate secondo tre strategie di selezione delle feature (all, best 3, worst 3) con intensità variabili dallo 0% al 95%. Per garantire la significatività statistica dei risultati, l'intera campagna sperimentale è stata condotta su 20 iterazioni indipendenti per ogni configurazione.

L'analisi comparativa evidenzia il Label Flipping come la minaccia più critica, capace di indurre una degradazione delle performance per tutti i modelli già a livelli medi di corruzione. Al contrario, la Random Forest si è distinta per una superiore robustezza strutturale, mitigando efficacemente gli attacchi basati sulla corruzione delle feature. Altre vulnerabilità, come il Top-K Drop e il rumore gaussiano, hanno mostrato impatti differenziati, colpendo in modo più incisivo il modello MLP.

In conclusione, lo studio sottolinea l'importanza della selezione di architetture resilienti e del monitoraggio dell'integrità dei dati in contesti critici come quello bio-medico.

Il codice sorgente e la documentazione tecnica del progetto sono disponibili al seguente indirizzo: https://github.com/LorenzoPinaUnimib/Dataset_Degradation.

1. Introduzione e Contesto del Progetto

1.1 Motivazione: Perché Studiare il Data Poisoning

Il Data Poisoning, o avvelenamento dei dati, rappresenta una delle minacce più insidiose per i sistemi di machine learning in produzione.

In ambito clinico, le fonti di corruzione dei dati sono molteplici e spesso difficilmente evitabili: errori umani nell'etichettatura dei campioni, variabilità nella misurazione delle caratteristiche citologiche, malfunzionamenti degli strumenti di acquisizione, manipolazioni intenzionali dei dati, migrazione e integrazione di dataset eterogenei e obsolescenza delle misurazioni nel tempo (data drift).

Comprendere come e quanto velocemente un modello degradi al variare del tipo e dell'intensità di corruzione è dunque fondamentale non solo da una prospettiva teorica, ma anche per la progettazione di sistemi diagnostici robusti, per la definizione di soglie di qualità dei dati accettabili, e per la selezione dell'architettura di modello più resiliente per uno specifico contesto applicativo.

1.2 Obiettivi della Ricerca Estesa

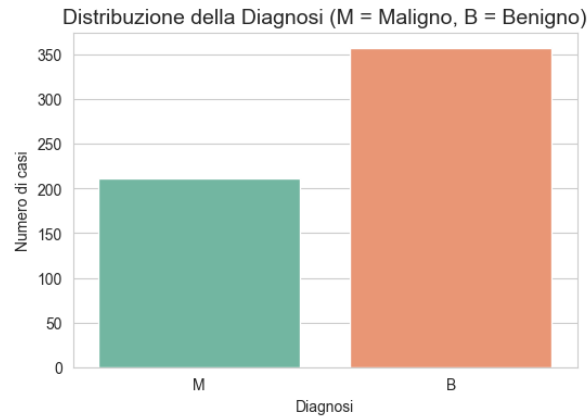
La ricerca estesa si articola su quattro obiettivi principali:

- Implementare una pipeline sistematica di Data Poisoning che permetta di applicare in modo controllato e riproducibile varie tipologie di corruzione a diverse intensità.
- Valutare quantitativamente l'impatto di ciascun tipo di corruzione sulle performance di tre famiglie di modelli (SVM, Random Forest, MLP), misurando Accuracy, Precision, Recall e F1-Score.
- Analizzare l'interazione tra strategia di selezione delle feature (all, best 3, worst 3) e resilienza alla corruzione, determinando se operare su sottoinsiemi di feature modifichi la vulnerabilità dei modelli.
- Identificare i metodi di corruzione più critici, le soglie di deterioramento significativo delle performance, e i modelli più robusti, per fornire raccomandazioni pratiche per l'implementazione clinica.

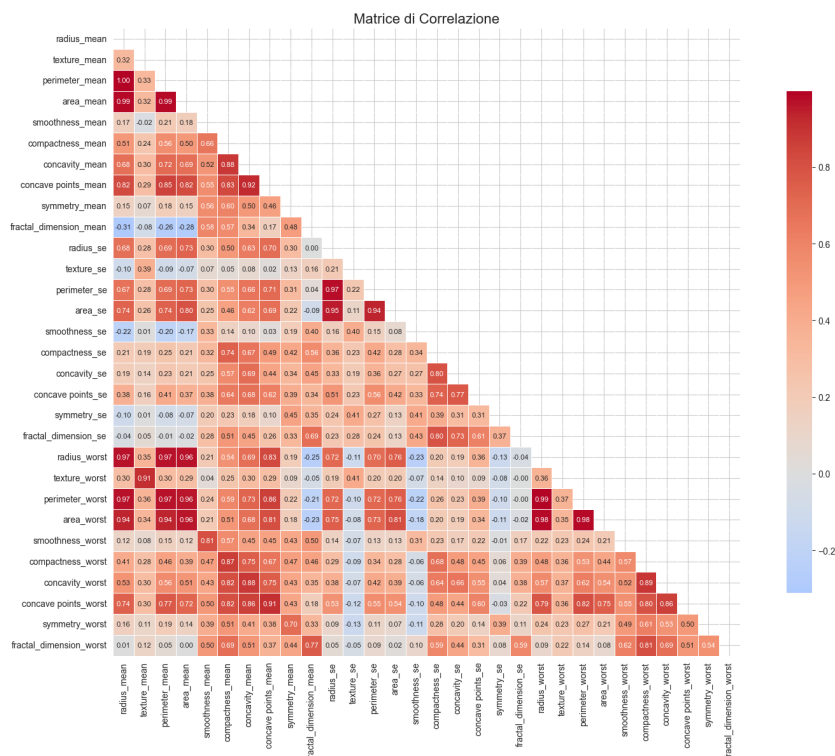
2. Dataset e Preprocessing di Base

2.1 Wisconsin Breast Cancer Dataset: Riepilogo

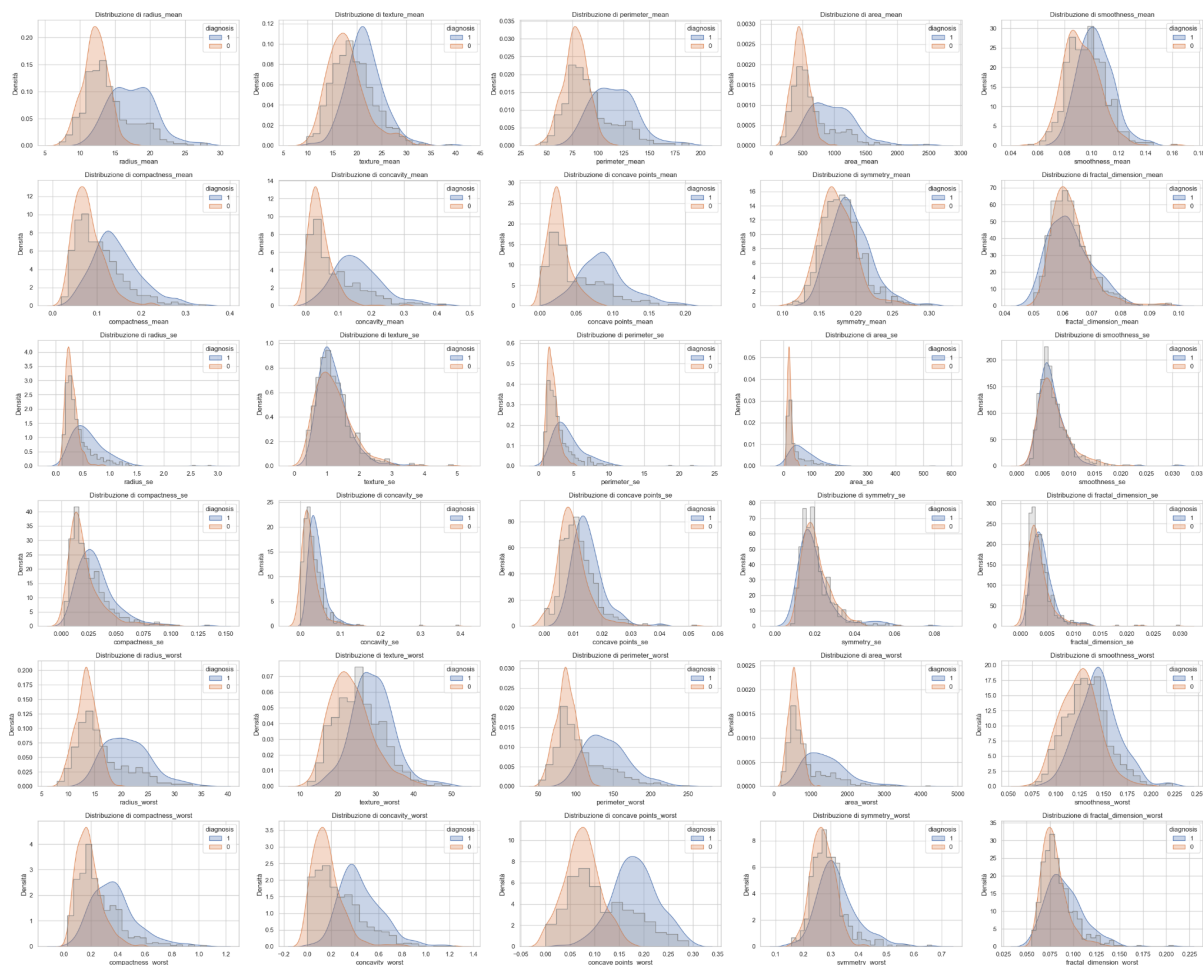
Il Wisconsin Breast Cancer (Diagnostic) Dataset, acquisito tramite Kaggle Hub, rappresenta il punto di partenza del progetto. Si tratta di un benchmark consolidato nella letteratura di machine learning biomedico, caratterizzato da 569 osservazioni citologiche e 30 feature predittive continue estratte da immagini digitalizzate. La variabile target è binaria: M (Maligno, 212 casi, 37.3%) e B (Benigno, 357 casi, 62.7%).



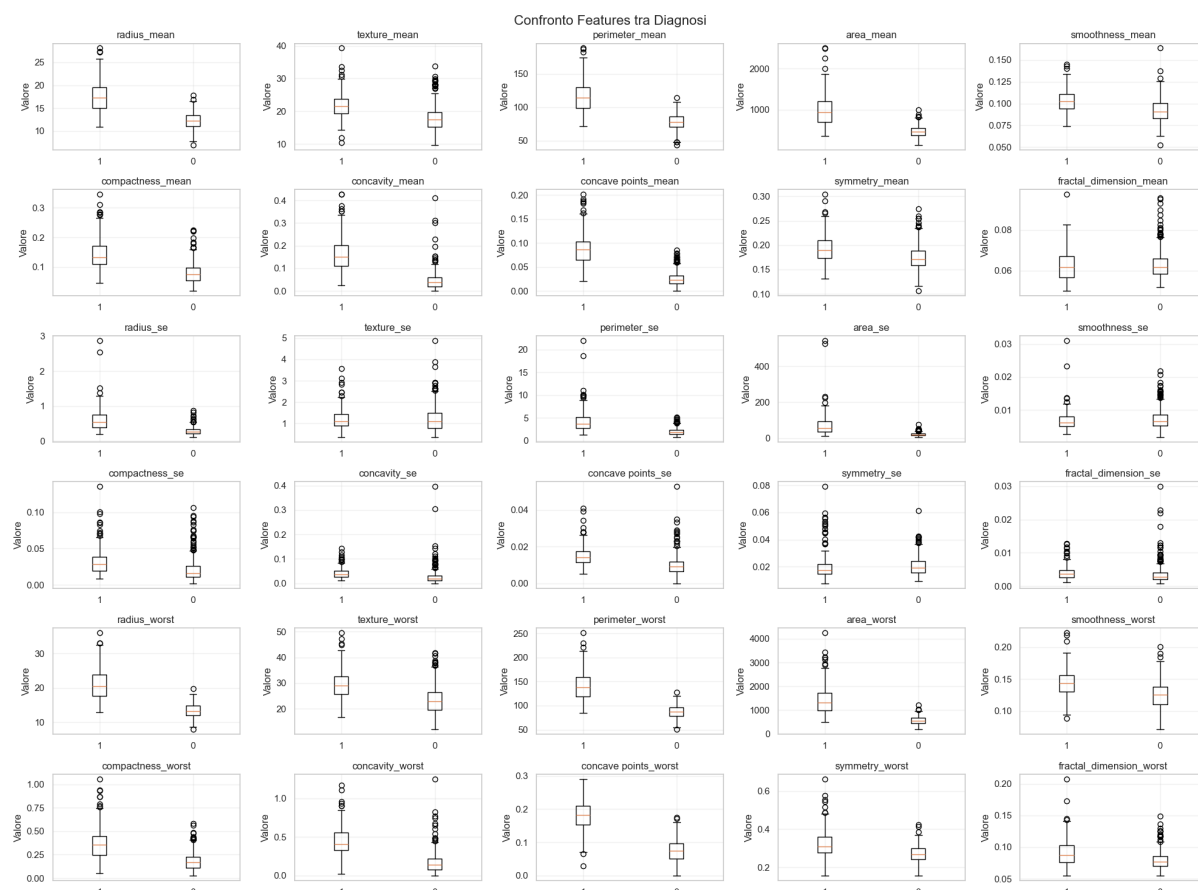
Le 30 feature coprono dieci misurazioni morfologiche fondamentali — raggio, texture, perimetro, area, smoothness, compattezza, concavità, punti concavi, simmetria, dimensione frattale — ciascuna rappresentata da tre statistiche: media (_mean), errore standard (_se) e valore peggiore (_worst), dal quale si ricava un'alta correlazione tra dati.



Analizzando la distribuzione delle feature all'interno del Dataset si nota come le medie e le worst sono distribuite secondo una distribuzione normale e nelle diagnosi maligne i valori tendono ad essere più elevati. Le deviazioni standard sono mediamente vicine allo 0.



L'analisi degli outliers attraverso boxplot ha rivelato la presenza di valori anomali, particolarmente evidenti nelle feature di texture e dimensione frattale. Questi outliers sono stati mantenuti nell'analisi in quanto potenzialmente rappresentativi di casi clinici limite e per preservare la già modesta dimensione campionaria disponibile.



2.2 Preprocessing Iniziale e Split Train/Test

Dopo la rimozione della colonna non informativa id, la codifica della variabile target tramite LabelEncoder ($M \rightarrow 1$, $B \rightarrow 0$) e la verifica dell'assenza di valori mancanti e duplicati, il Dataset viene suddiviso in training set e test set con un rapporto 80/20 con `random_state=42`. La stratificazione garantisce il mantenimento delle proporzioni delle classi in entrambi i subset.

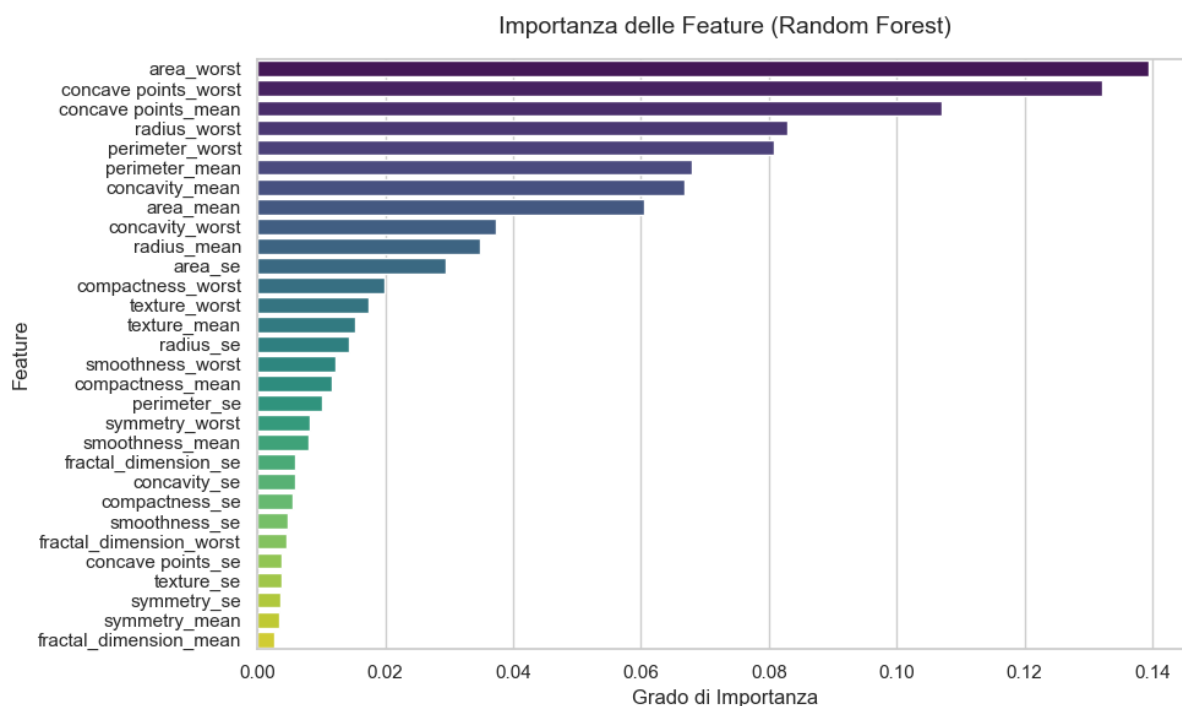
La standardizzazione viene applicata tramite StandardScaler: lo scaler viene fittato esclusivamente sui dati di training e applicato sia al training che al test set, prevenendo data leakage. Questo schema di standardizzazione viene replicato per ogni versione corrotta del dataset, garantendo che la corruzione venga applicata prima della normalizzazione e che lo scaler si adatti alla distribuzione corrotta del training set.

2.3 Architettura della Pipeline Sperimentale

La pipeline sperimentale adottata introduce una struttura a tre livelli annidati rispetto alla baseline originale.

Il primo livello riguarda la modalità di selezione delle feature (all features, best 3 features, worst 3 features). Le “best” e “worst” features sono estrapolate utilizzando un RandomForestClassifier temporaneo dalla funzione `_get_selected_features()`, eseguito sul

dataset pulito per poter impattare le colonne più e meno importanti del dataset originale con i vari metodi di sporcamento.



Il secondo livello copre il tipo di corruzione (Label Flipping, rumore gaussiano, introduzione di outlier, introduzione di valori mancanti, Data Drift, drop randomico di feature, azzeramento delle feature migliori e introduzione di duplicati).

Il terzo livello definisce l'intensità di corruzione (19 livelli da 0.05 a 0.95 con step 0.05, più la baseline pulita a livello 0).

Per ognuna di queste combinazioni vengono addestrati e valutati tre modelli (SVM, Random Forest, MLP), per un totale di $3 \times 8 \times 20 \times 3 \times 20 = 28.800$ esperimenti elementari, aggregati in 1.440 punti dati medi dopo averaging sui 20 run.

3. Metodi di Avvelenamento del Dataset

Il nucleo del presente lavoro risiede nell'implementazione sistematica di otto distinte strategie di data poisoning, tutte basate sulla libreria Python Pucktrick:

1. Label Flipping
2. Gaussian Noise
3. Outlier Injection
4. Missing Features
5. Data Drift
6. Random Feature Drop
7. Top-K Drop
8. Duplicates

La scelta di una libreria dedicata garantisce coerenza implementativa tra i diversi metodi e riproducibilità degli esperimenti. Ogni funzione di corruzione è integrata in un dispatcher centralizzato (`CORRUPTION_FNS`) che gestisce in modo uniforme la separazione tra feature matrix X e vettore target y .

3.1 Label Flipping (Avvelenamento dell'Etichetta)

Il label flipping rappresenta la forma più diretta di avvelenamento del training set: una frazione `noise_ratio` delle etichette diagnostiche viene invertita casualmente, trasformando diagnosi benigne in maligne e viceversa. Dal punto di vista implementativo il vettore y viene passato al modulo `pucktrick.labels` con `selection_criteria="all"` e `mode="new"`. Il parametro `percentage` corrisponde direttamente al `noise_ratio` testato.

Questo tipo di attacco simula scenari reali di: errori di refertazione da parte di patologi (falsi positivi/negativi nell'etichettatura manuale), corruzione intenzionale dei dati in attacchi maliziosi pianificati, migrazione di dati con mapping errato tra sistemi diagnostici diversi. È importante notare che il label flipping non dipende dalla modalità di selezione delle feature, ma le etichette sono indipendenti dal sottoinsieme di feature considerato; pertanto la stessa curva di degradazione si applica a tutte e tre le modalità. Questo aspetto è esplicitamente gestito nel codice tramite il metodo `plot_label_flipping_only()` che visualizza un unico grafico globale per questo tipo di attacco.

3.2 Gaussian Noise (Rumore Gaussiano sulle Feature)

L'iniezione di rumore gaussiano agisce sulle feature numeriche del dataset di training: per le colonne selezionate, viene aggiunto un disturbo distribuito normalmente con parametri determinati dalla libreria Pucktrick (`distribution="random"`). La funzione `add_gaussian_noise()` implementa una logica a due stadi: prima seleziona il sottoinsieme di feature candidate tramite `_get_selected_features()`, poi determina quante di esse corrompere in funzione del `noise_ratio`.

Il Gaussian Noise simula variabilità strumentale nelle misurazioni citologiche, errori di quantizzazione nell'acquisizione delle immagini digitali, variabilità inter-laboratorio nelle

procedure di preparazione dei campioni e rumore intrinseco nei processi di segmentazione automatica.

3.3 Outlier Injection (Iniezione di Valori Anomali)

L'iniezione di outlier introduce valori estremi nelle feature selezionate: la funzione `add_outliers()` passa la strategia al modulo `pucktrick.outliers` che, per una frazione `outlier_ratio` delle osservazioni, sostituisce i valori originali con valori anomali generati secondo criteri interni alla libreria. Analogamente al `gaussian noise`, la selezione delle feature candidate segue le modalità "all", "best 3" o "worst 3".

Gli outlier rappresentano una categoria di corruzione qualitativamente diversa dal rumore gaussiano: mentre il rumore gaussiano è distribuito uniformemente su tutte le osservazioni, gli outlier colpiscono un sottoinsieme di osservazioni con disturbi di grande entità.

Scenari reali modellati dalla Outlier Injection includono: misurazioni su campioni contaminati o degradati, errori di battitura o di unità di misura nei sistemi di archiviazione e casi clinici eccezionali che non rispecchiano la popolazione target.

3.4 Missing Features (Valori Mancanti)

Il quarto metodo introduce valori mancanti (NaN) nelle feature selezionate tramite il modulo `pucktrick.missing`. La funzione `add_missing_features()` applica una strategia di sostituzione immediata dei valori mancanti con il valore zero:

```
X_corrupted[affected_cols] = X_corrupted[affected_cols].fillna(0).
```

La scelta dell'imputazione a zero — anziché strategie più sofisticate come *media/media* o *k-NN imputation* — è intenzionale e rispecchia uno scenario pessimistico in cui i dati mancanti vengono rimpiazzati con un valore di default fisso senza considerazione del contesto.

I valori mancanti sono estremamente comuni in contesti clinici reali: risultati di esami non disponibili, rifiuto del paziente a determinati test, guasti tecnici degli strumenti, difetti nei processi di digitalizzazione dei referti storici.

L'imputazione a zero in questo contesto simula un sistema informativo che registra l'assenza del dato con un placeholder numerico (spesso usato in sistemi legacy), introducendo un bias sistematico verso valori bassi per le feature interessate.

3.5 Data Drift (Deriva dei dati)

Il Data Drift simula una variazione sistematica della distribuzione delle feature nel tempo. La funzione `add_data_drift()` utilizza il modulo `pucktrick.noise` configurato con `distribution="shift"`, applicando uno spostamento controllato alle feature selezionate. Come per *Gaussian Noise* e *Outlier Injection*, il sottoinsieme di feature interessate viene determinato tramite `_get_selected_features()` secondo le modalità "all", "best 3" o "worst 3". Il parametro `drift_ratio` definisce la percentuale di osservazioni da alterare.

A differenza del rumore gaussiano, il Data Drift modifica sistematicamente i dati aggiungendo o sottraendo una quantità costante. Nel codice, tale quantità è definita da "shift_value":1.0 e "shift_unit": "std", il che significa che ogni valore selezionato viene traslato di una deviazione standard della feature corrispondente, in direzione casuale, dettata da "shift_sign": "random". L'operazione viene applicata in modo vettoriale a tutte le osservazioni selezionate, preservando la forma generale della distribuzione ma modificandone la posizione.

Questo tipo di corruzione rappresenta uno scenario particolarmente realistico nei sistemi di machine learning schierati in produzione, poiché non corrisponde a errori puntuali nei dati ma a cambiamenti progressivi del processo generatore dei dati. Esempi concreti includono: aggiornamenti o ricalibramenti degli strumenti diagnostici che modificano sistematicamente le misurazioni, variazioni nelle procedure di acquisizione delle immagini mediche, cambiamenti nelle caratteristiche demografiche della popolazione di pazienti nel tempo, differenze tra ospedali o laboratori che adottano protocolli diagnostici differenti e fenomeni di evoluzione temporale (dataset shift) tra il periodo di addestramento e quello di utilizzo operativo del modello.

3.6 Random Feature Drop (Rimozione casuale di features)

Il Random Feature Drop simula la perdita completa di informazione proveniente da una o più feature del dataset selezionate tramite le tre modalità precedentemente descritte. Tra le colonne candidate, una frazione drop_ratio di queste viene scelta casualmente e sottoposta a corruzione.

L'implementazione sfrutta il modulo `pucktrick.missing`, configurato con `percentage=1.0`, in modo che tutte le osservazioni delle feature selezionate vengano trasformate in valori mancanti (NaN). In una fase successiva, i valori mancanti vengono immediatamente sostituiti con il valore costante zero mediante l'istruzione `fillna(0.0)`.

Questa forma di corruzione rappresenta uno scenario più severo rispetto alla semplice introduzione di valori mancanti distribuiti casualmente. Nel caso del Missing Features, soltanto una parte delle osservazioni risulta incompleta; nel Random Feature Drop, invece, l'intera feature diventa inutilizzabile. L'attacco consente quindi di valutare quanto il modello dipenda da specifici attributi e quanto sia robusto alla perdita totale di sorgenti informative rilevanti.

Dal punto di vista applicativo, il Random Feature Drop riproduce situazioni reali quali il guasto permanente di un sensore o di uno strumento diagnostico, la mancata disponibilità di un intero esame clinico per una coorte di pazienti, l'eliminazione erronea di una colonna durante la migrazione dei dati o incompatibilità tra sistemi informativi che impediscono l'acquisizione di determinate variabili.

3.7 Top-K Drop (Rimozione di Features per importanza)

Il Top-K Drop è implementato in modo analogo al Random Feature Drop, dal quale si distingue nella selezione delle colonne.

Questo viene effettuato su una percentuale “drop_ratio” delle colonne del dataset, eliminando prima le più importanti sulla base dell'ordinamento stabilito da RandomForestClassifier.

Questa metodologia punta a rendere più diretto il processo di Random Feature Drop, portando ad un risultato che più drasticamente va a ridurre la significatività dei dati, con l'aumento del “drop_ratio”.

3.8 Duplicates (Duplicazione di osservazioni)

La duplicazione di record introduce nel Dataset copie esatte di osservazioni già esistenti, alterando la distribuzione empirica dei dati senza modificarne direttamente i valori. La funzione `add_pucktrick_duplicates()` utilizza il modulo nativo `pucktrick.duplicate`, operando congiuntamente su feature ed etichette secondo il parametro `duplicate_ratio`, il quale controlla la percentuale di record da replicare.

Per garantire che ogni osservazione duplicata mantenga la corretta associazione tra attributi e classe diagnostica, il dataset delle feature (X) e il vettore delle etichette (y) vengono temporaneamente unificati in un unico dataframe prima dell'applicazione dell'iniettore e successivamente separati al termine del processo.

L'iniettore seleziona un sottoinsieme di osservazioni dal Dataset originale e ne genera copie identiche, che vengono aggiunte al dataframe esistente mediante concatenazione. Di conseguenza, il numero complessivo di esempi aumenta, mentre il contenuto informativo del Dataset rimane invariato.

A differenza delle tecniche precedenti, che introducono rumore, valori anomali o dati mancanti, la duplicazione non altera la qualità dei singoli record ma modifica la frequenza relativa con cui determinate osservazioni compaiono durante il processo di addestramento. L'effetto principale consiste nell'introdurre un bias nella distribuzione campionaria, poiché alcuni esempi acquisiscono un peso maggiore rispetto ad altri. Questo può portare il modello a sovrastimare l'importanza di particolari pattern presenti nei dati duplicati, aumentando il rischio di overfitting e riducendo la capacità di generalizzazione verso nuovi campioni.

Dal punto di vista applicativo, la duplicazione di record riproduce situazioni realistiche quali l'importazione ripetuta degli stessi dati, fusioni di database contenenti osservazioni duplicate non correttamente identificate, sincronizzazioni multiple tra sistemi clinici differenti o procedure di raccolta dati che registrano accidentalmente più copie dello stesso esame. In ambito medico tali fenomeni possono alterare la rappresentatività statistica della popolazione analizzata, inducendo il modello ad apprendere distribuzioni distorte rispetto alla realtà operativa.

4. Architettura dei Modelli e Configurazione Sperimentale

4.1 Support Vector Machine (SVM)

Il modello SVM è configurato con kernel lineare (`kernel="linear"`), parametro di regolarizzazione `C=1.0` e seed dinamico per run. Viene utilizzato un kernel lineare per tre ragioni principali: analisi di un modello “semplice”, maggiore trasparenza interpretativa della risposta alla corruzione e maggiore efficienza computazionale.

Le SVM con kernel lineare trovano il margine massimo tra le classi in uno spazio lineare, rendendole teoricamente sensibili a Label Flipping (che sposta il confine di margine ottimale) e a outlier (che possono diventare Support Vectors con grande influenza sull'iperpiano).

Il parametro `C=1.0` rappresenta un bilanciamento moderato tra massimizzazione del margine e tolleranza degli errori di training.

4.2 Random Forest

Il Random Forest è configurato con `n_estimators=100` alberi decisionali e seed dinamico per run. Come modello ensemble basato su aggregazione di predittori deboli, il Random Forest presenta caratteristiche di robustezza intrinseche: il bagging e la selezione casuale delle feature ad ogni split distribuiscono l'impatto delle corruzioni tra i diversi alberi, prevenendo che singoli punti anomali dominino l'intera predizione. Questa proprietà rende il Random Forest un benchmark naturale di robustezza contro cui confrontare SVM e MLP.

4.3 MLP (Multi-Layer Perceptron)

Il modello MLP è configurato con `hidden_layer_sizes=(64, 32)`, `activation="relu"`, `alpha=0.0001` (L2 regularization), `early_stopping=True` con `validation_fraction=0.1`, `max_iter=1000` e seed dinamico per run.

La presente implementazione utilizza l'`MLPClassifier` di scikit-learn; questa scelta è dettata dal già alto tempo computazionale necessario per eseguire la pipeline.

L'early stopping con 10% di validation set interno introduce un elemento di adattamento dinamico: quando il dataset di training è corrotto, il modello potrebbe apprendere pattern fuorvianti che non si generalizzano al validation set interno, portando a un arresto anticipato. Questo meccanismo può fungere da barriera parziale contro l'overfitting sulla corruzione, a scapito però di una riduzione del numero effettivo di iterazioni di addestramento.

4.4 Configurazione dell'Esperimento e Metriche

Ogni modello viene re-inizializzato ad ogni run con `seed = 42 + run_index`, garantendo variabilità controllata tra le iterazioni. Le metriche di valutazione calcolate per ogni esperimento sono: Accuracy (frazione di predizioni corrette), Precision (precisione

nell'identificazione dei maligni), Recall (frazione di maligni correttamente identificati), e F1-Score (media armonica di Precision e Recall).

Il test set rimane invariato in tutti gli esperimenti — la corruzione è applicata esclusivamente al training set — garantendo che i miglioramenti o peggioramenti delle metriche riflettano genuinamente la variazione nella qualità del modello appreso e non cambiamenti nella distribuzione del target di valutazione.

5. Risultati Sperimentali

5.1 Performance Baseline (Senza Corruzione)

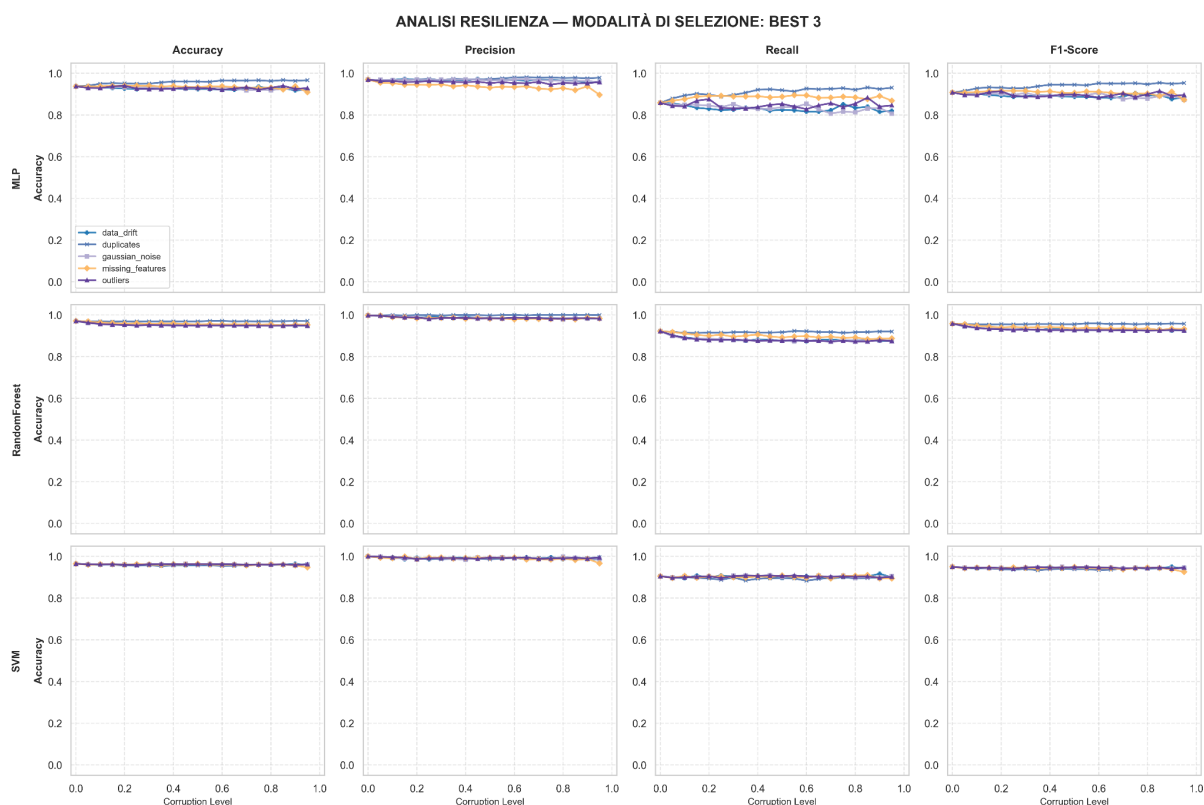
La performance di baseline, ricavata dalle diverse corruzioni a soglia 0% indica che i modelli sono in grado di rappresentare con qualità elevata il modello pulito, avendo una F1-Score di ~0.9 per la MLP e vicino ad 1 per Random Forest e SVM.

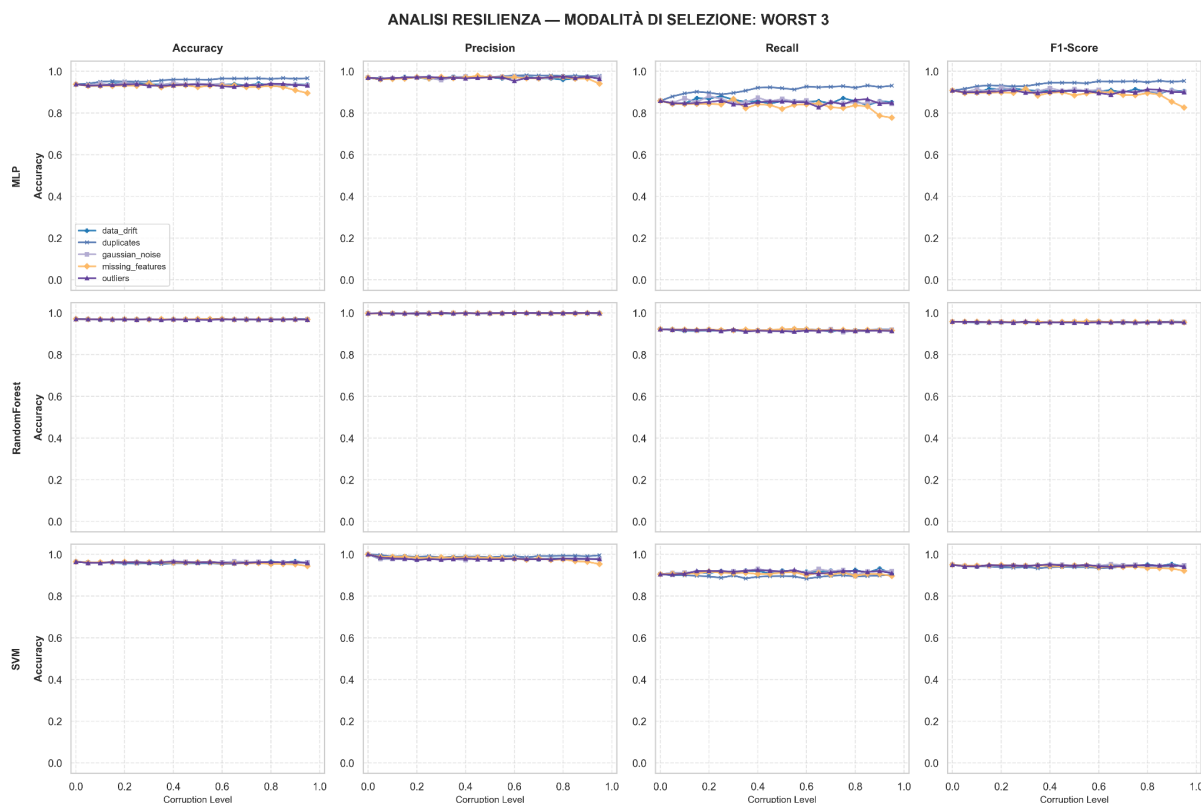
5.2 Analisi di resistenza per Best e Worst 3

Risulta immediatamente evidente che, per i modelli di corruzione soggetti all'analisi delle migliori e peggiori 3 feature, il degrado delle performance è irrisorio o sporadico, nel caso del Recall.

Questo è probabilmente dato da due motivi:

- Il ~10% delle feature non permette di rimuovere significatività sufficiente al dataset: la somma complessiva delle 3 feature più importanti si aggira attorno al ~36% dell'importanza complessiva.
- Il dataset è intrinsecamente correlato tra le features di costruzione, per via di dati duplicati tra `_mean`, `_worst` e `_se` e correlazione tra dati quali `perimeter` e `radius` (altamente visibile nella matrice di correlazione); è altamente probabile che i modelli riescano ad estrapolare comunque la corretta classificazione da una versione diversa del dato.



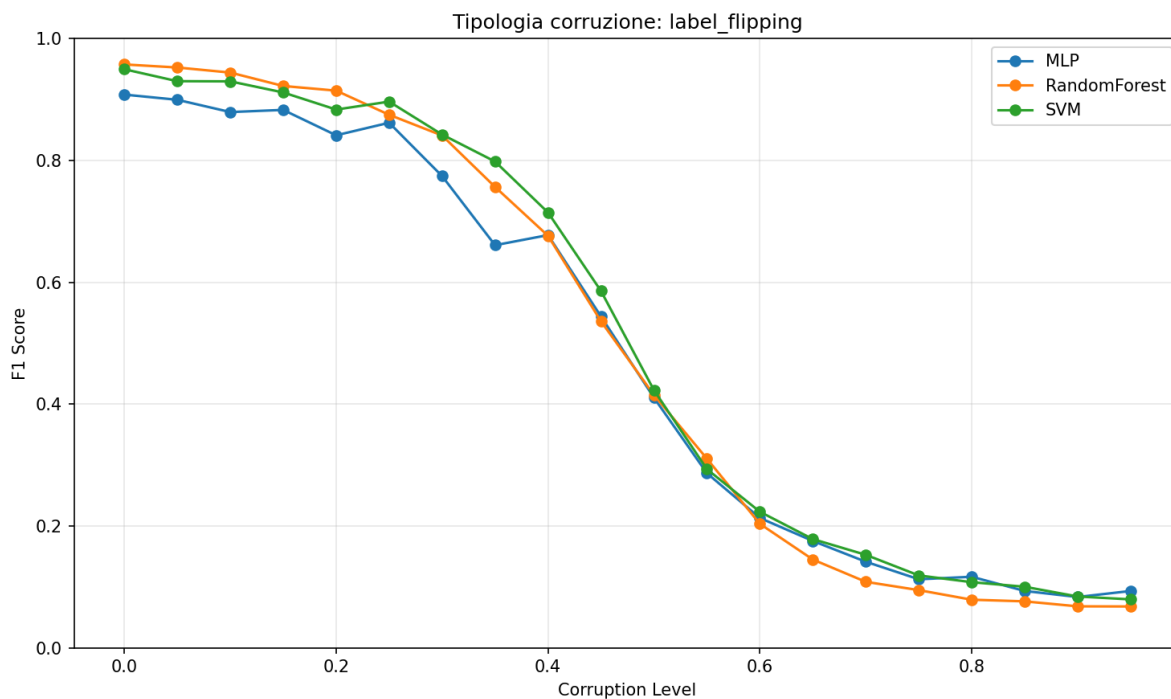


5.3 Impatto del Label Flipping

Il Label Flipping si conferma come l'attacco più devastante tra i metodi testati. La degradazione delle performance è immediata e drastica: già a un tasso di corruzione del 20%, tutti e tre i modelli mostrano un calo misurabile di F1-Score. A livelli di corruzione superiori al 40-50%, le performance convergono verso valori casuali ($F1 \approx 0.5$), indicando un'inversione completa della funzione di classificazione appresa.

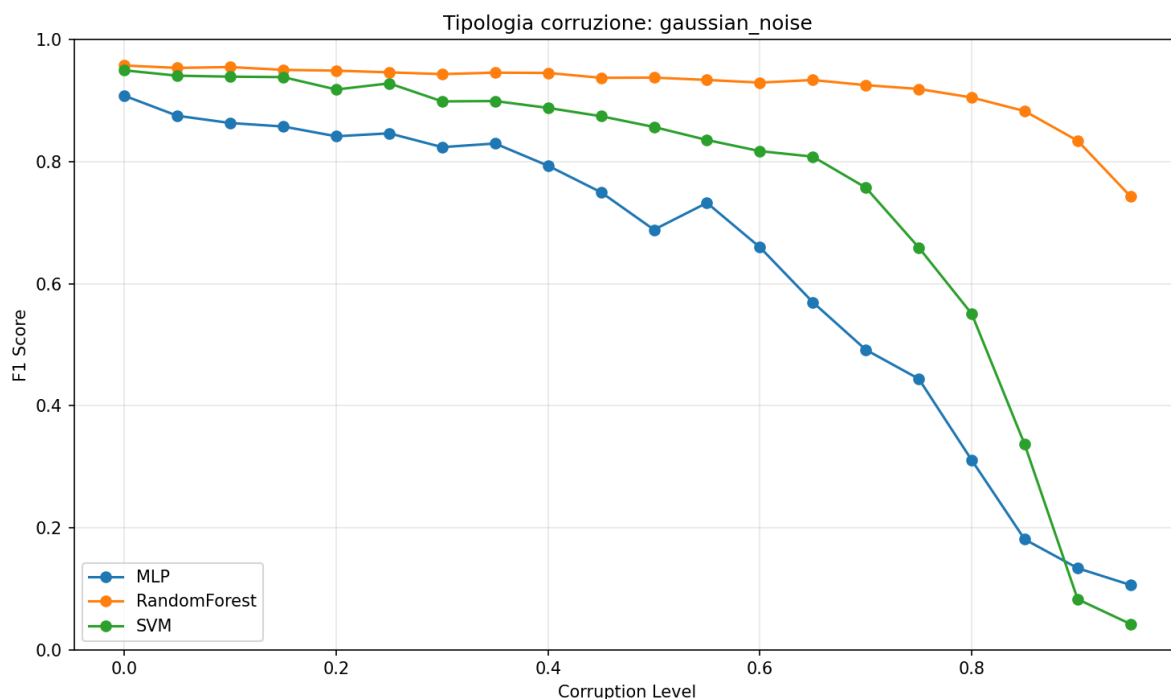
Questo comportamento è spiegabile teoricamente: con il 50% di Label Flipping, il Dataset contiene la stessa quantità di informazioni corrette e scorrette, rendendolo indistinguibile da un Dataset completamente casuale. Per livelli superiori al 50%, il modello "impara" la relazione inversa — classificando i maligni come benigni e viceversa — portando a un ribaltamento sistematico delle predizioni che può paradossalmente ridurre le performance a valori prossimi allo zero, peggio del caso casuale.

Un risultato di particolare interesse è che tutti e tre i modelli mostrano una degradazione quasi identica sotto Label Flipping, indipendentemente dalla modalità di selezione delle feature. Questo conferma che il Label Flipping è un attacco globale che colpisce la struttura stessa della supervisione, non la rappresentazione delle feature.



5.4 Impatto del Gaussian Noise

Il Gaussian Noise produce una degradazione più graduale e differenziata rispetto al label flipping; questa è lenta e progressiva: i modelli mantengono performance accettabili ($F1 > 0.8$) fino a livelli di rumore del 40-50%, con una caduta più ripida oltre quella soglia.



Il Random Forest dimostra la maggiore robustezza al gaussian noise tra i tre modelli, grazie alla sua natura ensemble: il rumore su una singola feature influenza solo gli alberi che la utilizzano come split al nodo corrispondente, mentre gli altri alberi mantengono la loro

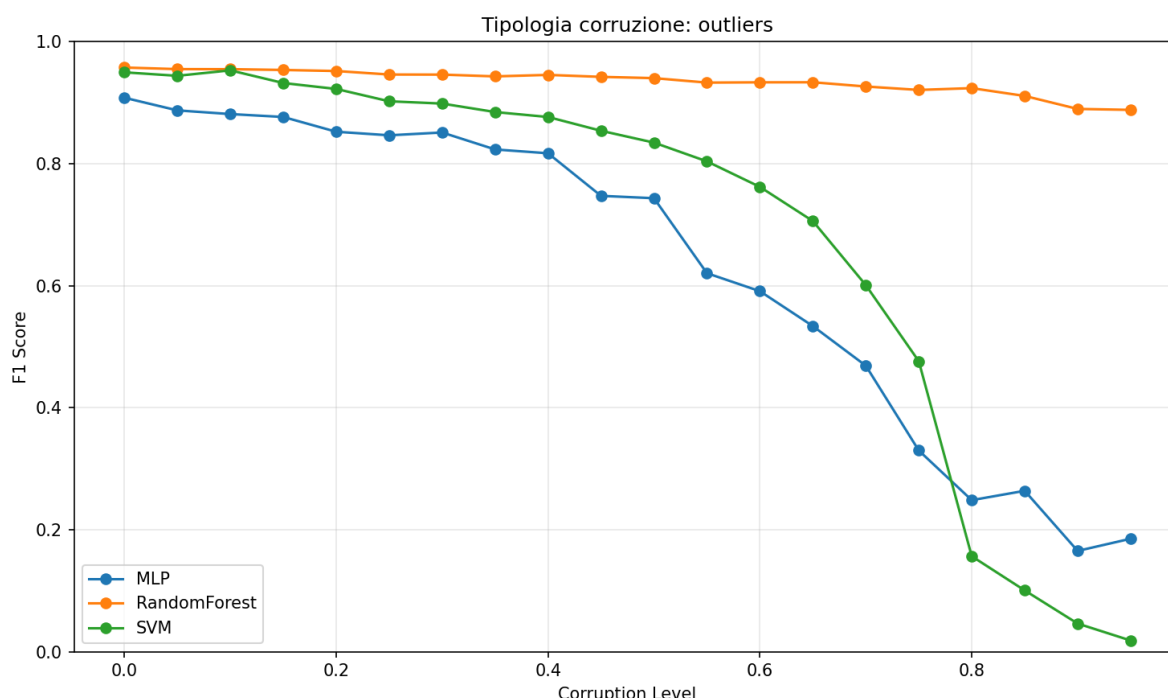
correttezza. SVM non riesce a distinguere propriamente le categorie solo con percentuali di corruzione alte, mentre il MLP va in crisi molto prima, probabilmente per via di overfitting.

5.5 Impatto della Outlier Injection

L'iniezione di outlier mostra un comportamento analogo al rumore, data la sua natura simile di alterazione numerica delle features, portando a cali di prestazioni solo dopo il 40-50% di corruzione.

È di nota come la Random Forest riesca ad adattarsi con minime perdite anche a valori molto elevati di outliers, grazie alla robustezza intrinseca al fenomeno, la quale permette di mantenere una F1-Score > 0.9 anche con dataset quasi completamente sporco.

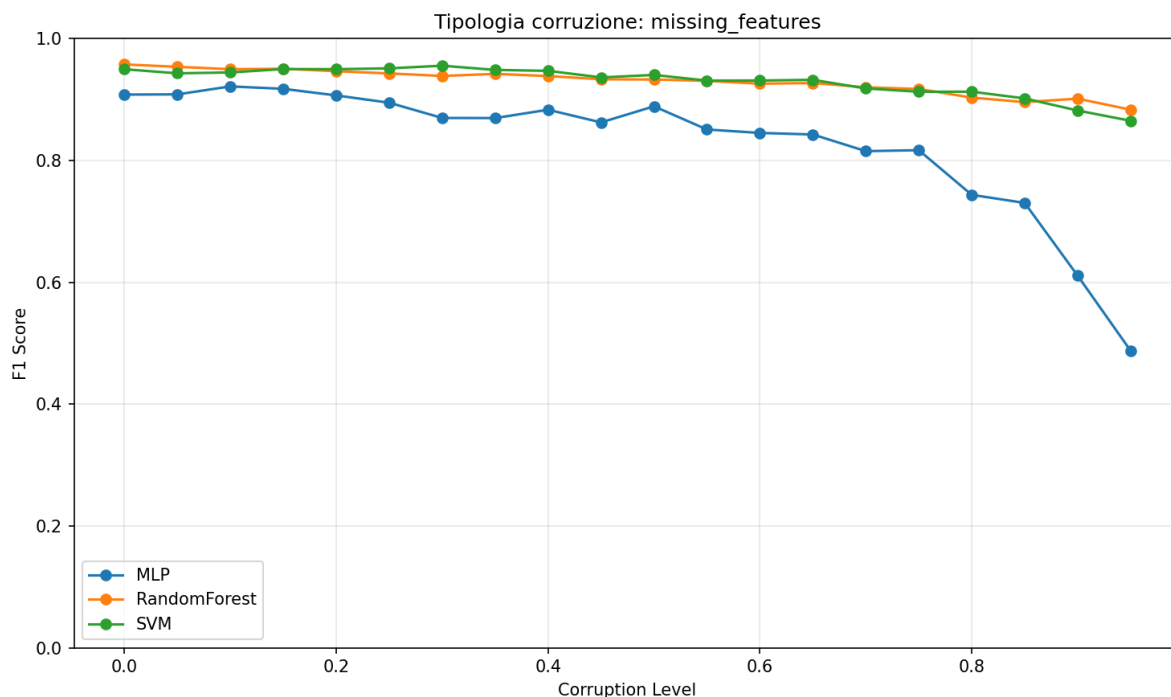
SVM e MLP, invece, hanno un crollo di forma simile al rumore, anche se marginalmente più ripido.



5.6 Impatto dei Valori Mancanti

I valori mancanti con imputazione a zero rappresentano, dei metodi testati, il secondo con il minor impatto complessivo sulle performance. Fino al 60% di missing values, le performance dei tre modelli rimangono stabili e quasi indistinguibili dalla baseline. Solo oltre il 60% di valori mancanti si registra un deterioramento significativo ($F1 < 0.85$) nell'MLP.

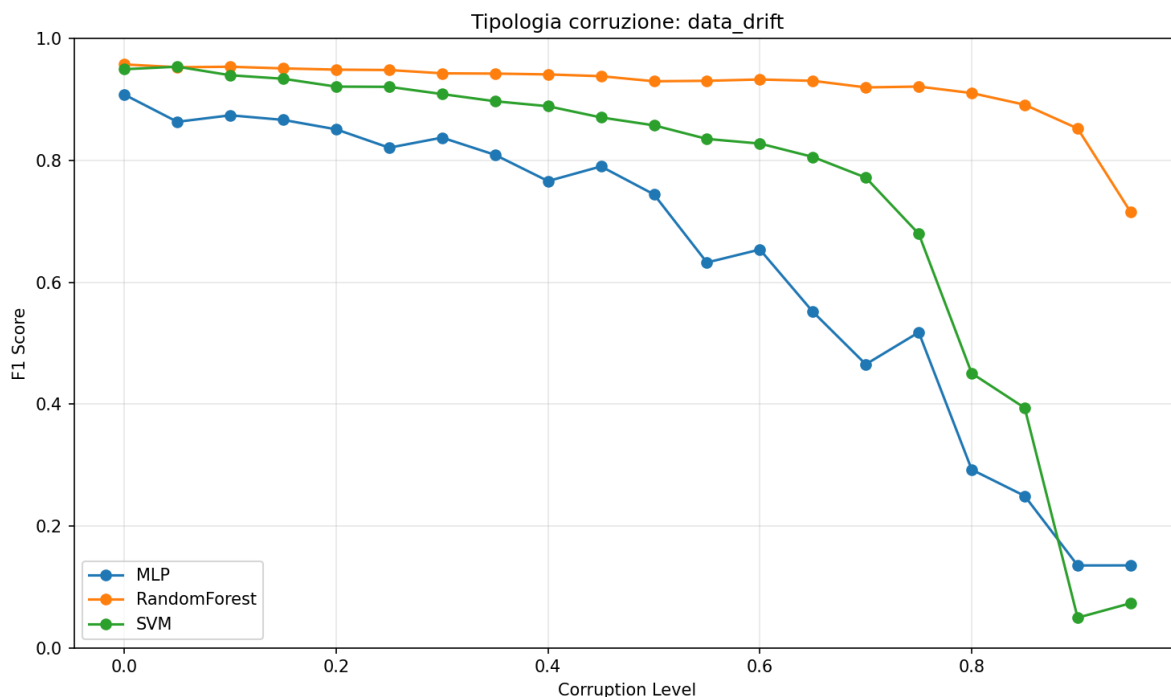
Questo risultato apparentemente sorprendente è spiegabile con due fattori: primo, l'imputazione a zero introduce un bias sistematico ma coerente (tutti gli esempi con valori mancanti vengono "spostati" nello stesso punto dello spazio delle feature, creando un cluster artificiale che i modelli possono comunque usare per l'apprendimento). Secondo, il Wisconsin Breast Cancer Dataset è caratterizzato da una forte separabilità lineare, per cui anche una rappresentazione degradata delle feature mantiene sufficiente discriminabilità.



5.7 Impatto del Data Drift

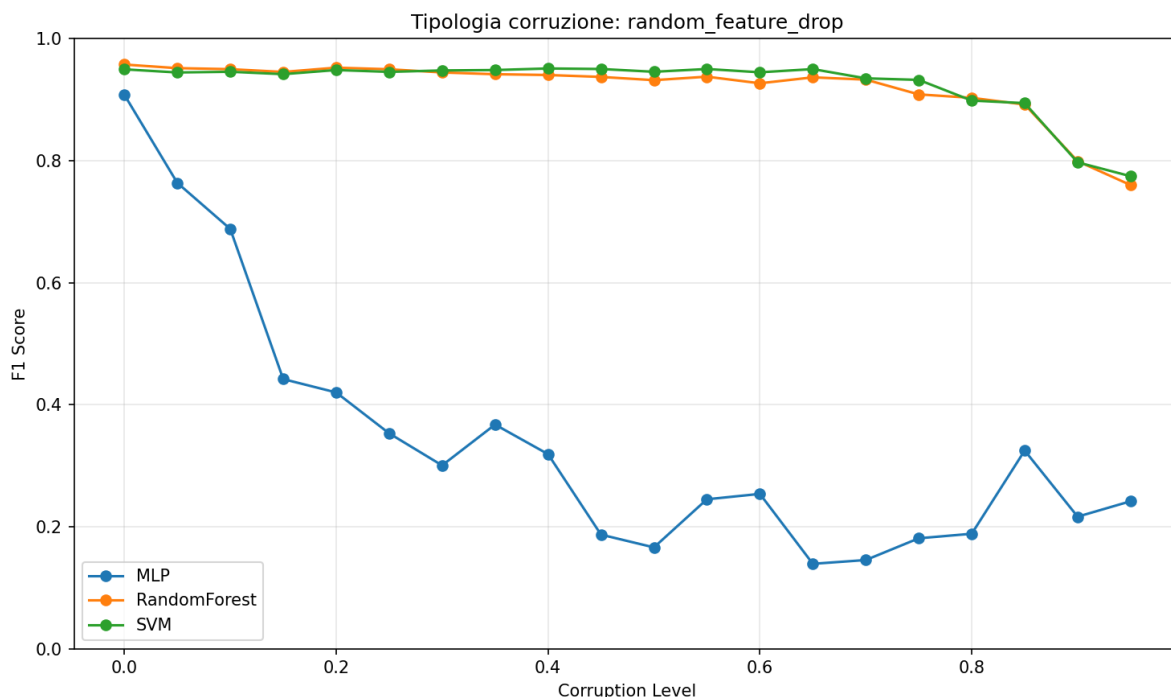
Il Data Drift introduce una lieve degradazione per MLP fino al 30% di corruzione, al di sopra del quale si nota una riduzione significativa dell'F1-Score sia per MLP che per SVM mentre, come già notato per le altre tipologie di corruzione, Random Forest riesce ad evitare crolli di performance fino a percentuali molto elevate.

Questo rispecchia un comportamento analogo agli altri tipi di sporcamento numerico dei dati.



5.8 Impatto del Random Feature Drop

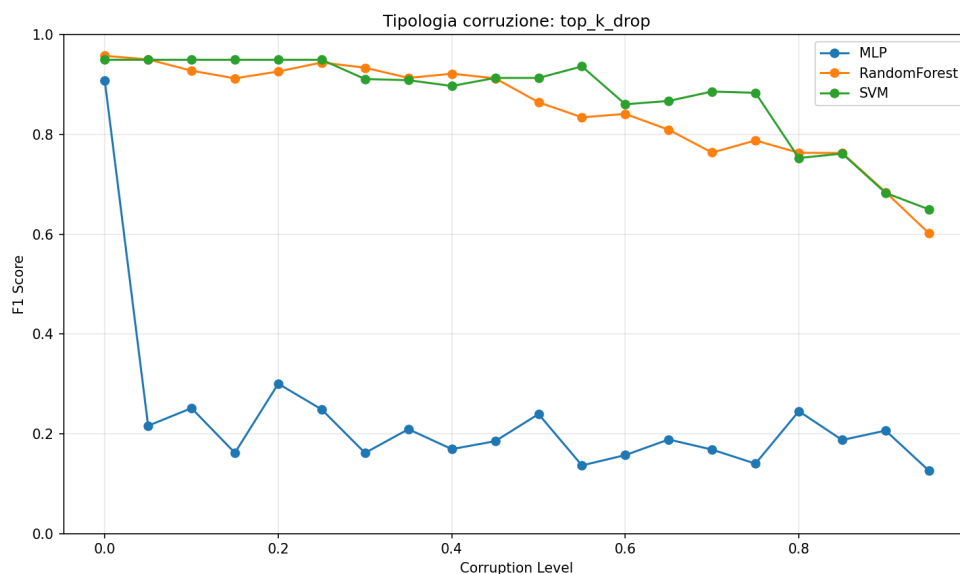
Il Random Feature Drop degrada molto velocemente le performance del Multilayer Perceptron mentre sia SVM che Random Forest resistono bene ad un drop randomico delle feature.



Questo è probabilmente dovuto da una discrepanza netta nei dati di train e di test, i quali differiscono per numero di colonne.

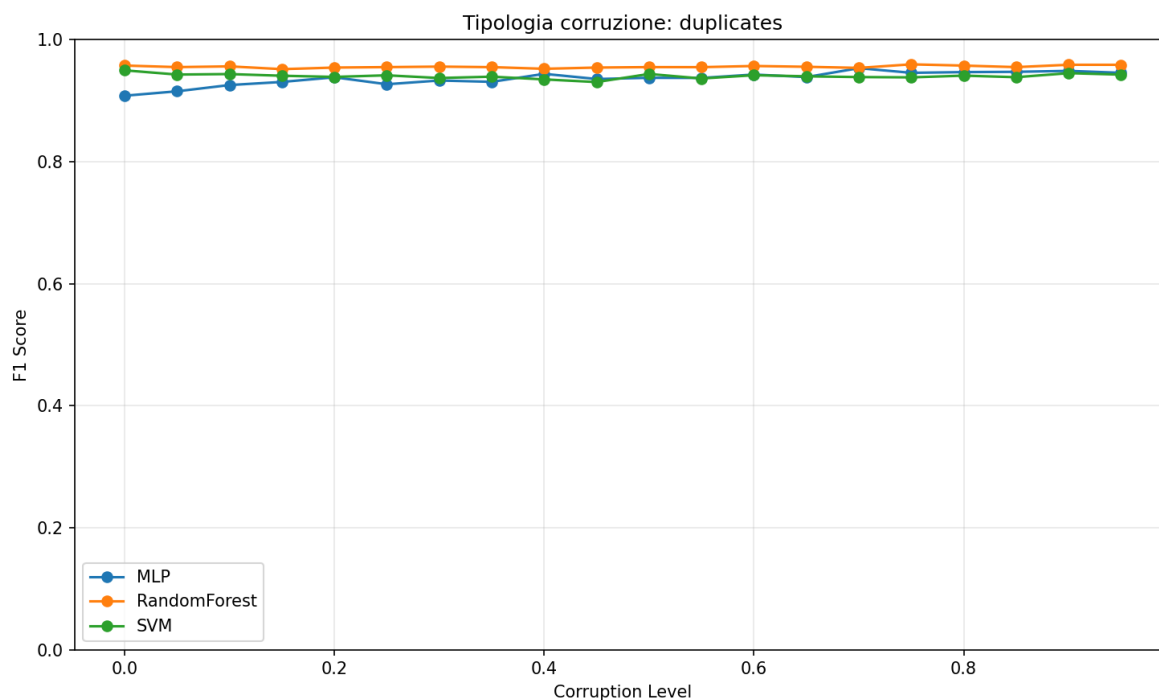
5.9 Impatto del Top-K Drop

Per natura, il Top-K Drop è analogo al Random Feature Drop, ma dato che perde per prime le colonne più significative, porta a peggiori performance già dal livello 0.05, andando a simulare un worst-case-scenario per Random Feature Drop.



5.10 Impatto dei Duplicati

Nell'aggiunta di dati duplicati, si può notare che tutti i modelli migliorano progressivamente con l'aggiunta di dati, a differenza di tutti gli altri metodi di sporco.



Anche se si possa supporre che l'aggiunta di dati duplicati porti ad overfitting, questa risulta in un aumento graduale di performance, dovuta in buona probabilità dalla dimensione limitata del dataset originale.

L'aggiunta di duplicati in questo caso fornisce ai modelli più tempo e dati sui quali essere addestrati; questo rispecchia anche quanto il Dataset risulti robusto e ben formato.

6. Analisi Comparativa dei Risultati

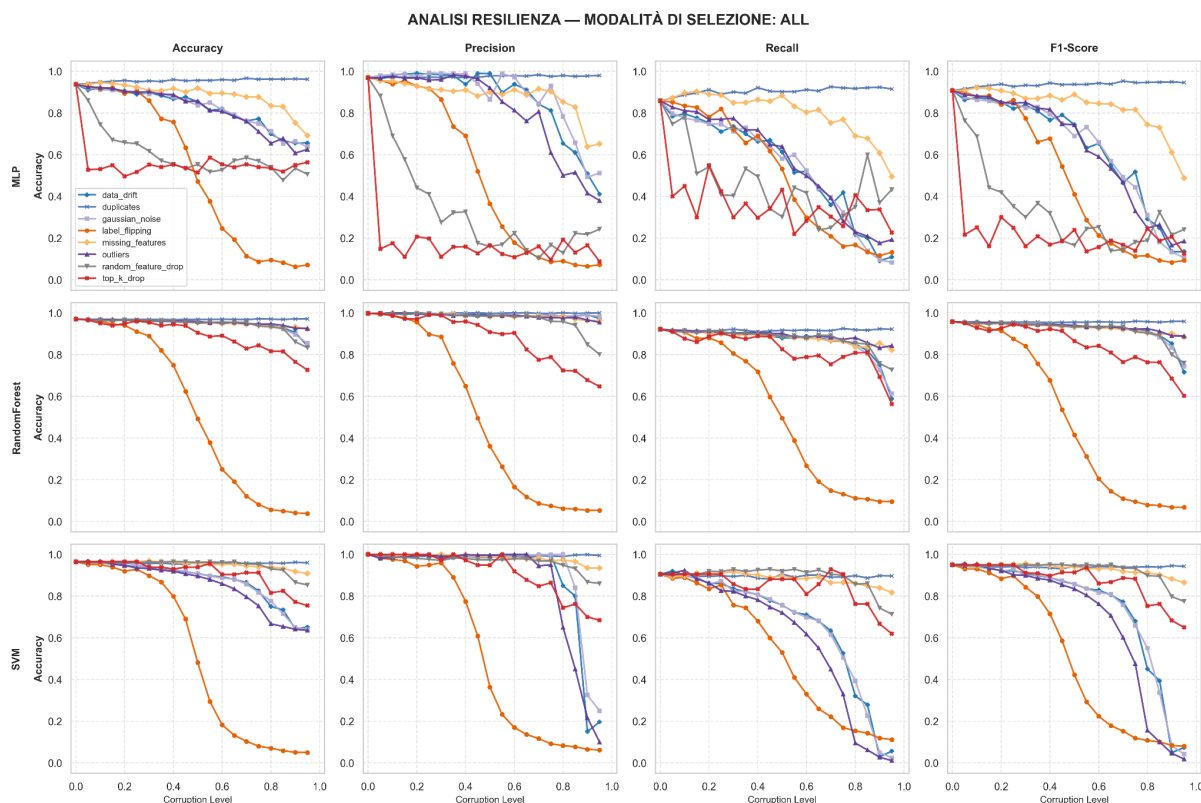
6.1 Gerarchia di Criticità dei Metodi di Corruzione

L'analisi comparativa degli otto metodi di corruzione permette di definire una gerarchia di criticità basata sulla velocità e sull'entità della degradazione:

- **Metodi Critici:** Il **Label Flipping** provoca una degradazione immediata e totale ($F1 \approx 0.5$ al 50% di corruzione). Il **Top-K Drop** rappresenta il peggiore scenario di feature loss, abbattendo le performance già a bassi livelli di intensità eliminando le variabili più discriminative.
- **Metodi ad Alto Impatto:** L'**Outlier Injection** e il **Random Feature Drop** mostrano una rapida degradazione, specialmente su modelli come MLP e SVM, che faticano a gestire la perdita di informazione strutturale o la presenza di valori estremi.
- **Metodi Moderati:** Il **Gaussian Noise** e il **Data Drift** portano a una degradazione progressiva, mantenendo performance accettabili fino a soglie medie (40-50%) prima di un crollo significativo.
- **Metodi a Basso Impatto:** Le **Missing Features** con imputazione a zero hanno un impatto limitato fino al 60-70% di corruzione, grazie alla resilienza del dataset alla perdita parziale di campioni.
- **Anomalie Positive:** I **Duplicates** costituiscono un caso unico, portando a un lieve miglioramento delle performance. Data la dimensione contenuta del dataset, la duplicazione funge da incremento del peso di campioni corretti durante l'addestramento.

6.2 Confronto tra Modelli: Robustezza Complessiva

L'analisi trasversale conferma la superiorità della **Random Forest** come modello più robusto in assoluto. La sua architettura ensemble mitiga efficacemente sia il rumore sulle feature che la perdita di colonne (drop). SVM e MLP seguono con una vulnerabilità decisamente superiore, in particolare verso gli attacchi che alterano la geometria dello spazio delle feature.



6.3 Effetto della Modalità di Selezione delle Feature sulla Robustezza

L'analisi per modalità di selezione delle feature rivela pattern differenziati in funzione del tipo di corruzione. Per il Gaussian Noise e gli outlier, la modalità "worst 3 features" mostra sistematicamente la maggiore robustezza relativa: corrompere le feature meno informative produce impatti minimi. La modalità "best 3 features" segue risultati analoghi alle peggiori, in quanto, anche se "migliori", le feature sporcate sono troppo basse rispetto al dataset complessivo.

La modalità "all features", invece, presenta degradazione di performance variegata rispetto al metodo di sporcamento, che rispecchia le caratteristiche dei modelli e del dataset stessi.

6.4 Analisi della Stabilità Statistica (20 Run)

Le 20 run indipendenti evidenziano che all'aumentare dell'intensità di corruzione aumenta sensibilmente la varianza dei risultati. I modelli diventano instabili, riflettendo la dipendenza dal campionamento casuale del disturbo quando l'informazione utile nel training set diventa scarsa.

Il Label Flipping mostra la crescita di variabilità più drastica, con intervalli di confidenza molto ampi tra il 30% e il 60% di corruzione. Questo riflette il fatto che, a quei livelli, piccole differenze nell'esatto sottoinsieme di Label Flipped (dovute ai diversi seed) possono portare a modelli con comportamenti molto diversi. Il Random Forest mostra la deviazione standard più contenuta anche sotto corruzione, confermando la sua maggiore stabilità statistica.

7. Discussione e Implicazioni

7.1 Relazione rispetto alla baseline

Sotto corruzione si osserva un'inversione delle performance rispetto alla baseline: se nel dataset pulito SVM e MLP possono eccellere, in scenari degradati (Gaussian Noise, Outlier, Drift) la Random Forest domina costantemente. Questo suggerisce che la scelta del modello per il deployment debba privilegiare la robustezza strutturale rispetto alla precisione assoluta su dati ideali.

7.2 Implicazioni per il Deployment Clinico

In ambito clinico, l'impatto devastante del Label Flipping e del Top-K Drop impone un monitoraggio rigoroso della qualità del dato. Non è sufficiente valutare il modello sulla baseline; è necessario testare la resilienza del sistema a possibili derive o errori sistematici di acquisizione prima dell'uso diagnostico.

In secondo luogo, la scelta del modello dovrebbe essere guidata non solo dalle performance baseline ma anche dalla robustezza attesa nel contesto specifico. Se il rischio principale è un'alta variabilità strumentale (simulata dal Gaussian Noise), il Random Forest rappresenta la scelta più sicura. Se il rischio è legato a errori di refertazione sistematici (Label Flipping), nessun modello è intrinsecamente più robusto degli altri: in questo caso, meccanismi di verifica della qualità delle etichette diventano fondamentali.

Infine, i risultati suggeriscono che soglie di tolleranza del 10-15% di corruzione possono essere accettabili per la maggior parte degli attacchi alle feature, ma anche solo il 5-10% di Label Flipping può già compromettere significativamente le performance. Le procedure di quality assurance per l'etichettatura dei dati dovrebbero pertanto essere particolarmente stringenti.

7.3 Limitazioni dello Studio

Lo studio è limitato dalla specificità dell'imputazione a zero per i valori mancanti e dalla natura statica del test set. In scenari reali, l'impiego di tecniche di imputazione avanzate potrebbe alterare i profili di robustezza osservati.

In secondo luogo, l'imputazione a zero per i valori mancanti rappresenta una strategia sub-ottimale e volutamente pessimistica. Strategie più sofisticate (mean imputation, k-NN imputation, MICE) avrebbero probabilmente mitigato significativamente l'impatto dei missing values, rendendo i risultati meno generalizzabili a contesti con imputazione avanzata.

In terzo luogo, per limitazioni di tempo e potenza computazionale, solo un semplice modello MLP è stato utilizzato al posto di una propria rete neurale convoluzionale; questa, data una sufficiente complessità potrebbe essere in grado di adattarsi a sporco nei dati cogliendo metodi alternativi per effettuare una corretta classificazione.

Infine, per evitare un'esplosione dal punto di vista del costo computazionale, lo studio si è limitato ad effettuare un pre-processing dei dati con StandardScaler, tralasciando altre

tecniche come RobustScaler o PCA, che potrebbero portare ad una mitigazione della degradazione per i dataset in partenza.

8. Conclusioni e prospettive future

8.1 Conclusioni

La presente ricerca ha fornito una valutazione sistematica della resilienza dei modelli di machine learning — SVM, Random Forest e MLP — di fronte a diverse forme di Data Poisoning applicate al Wisconsin Breast Cancer Dataset.

Abbiamo confermato che, mentre la baseline di prestazione è elevata per tutti i modelli, la loro capacità di generalizzazione diverge drasticamente in presenza di dati corrotti. Il Label Flipping si è rivelato il metodo più critico, capace di compromettere le performance in modo quasi totale, rendendo evidente che nessun modello può compensare sistematicamente etichette errate. Al contrario, la Random Forest si è dimostrata il modello più robusto, evidenziando come le tecniche di ensemble learning offrano una protezione intrinseca contro il rumore e la perdita di informazioni sulle feature.

8.2 Prospettive Future

I risultati ottenuti aprono diverse direzioni per ricerche future:

- **Mitigazione Attiva:** Oltre all'analisi della vulnerabilità, lo studio può essere esteso allo sviluppo di tecniche di Data Sanitization e Adversarial Training per ripulire attivamente i dataset prima dell'addestramento.
- **Modelli Avanzati:** Sarebbe opportuno sostituire l'MLP utilizzato con architetture di Deep Learning più complesse o modelli basati su Attention Mechanisms per verificare se la capacità di estrazione delle feature sia più resiliente alla corruzione.
- **Preprocessing Adattivo:** L'integrazione di tecniche di Robust Scaling e algoritmi di imputazione avanzati (come MICE o KNN-Imputation) potrebbe mitigare l'impatto di outlier e dati mancanti, offrendo una visione più chiara sulla reale tolleranza dei modelli.
- **Generalizzazione:** Testare l'efficacia di questi attacchi su dataset di dimensioni maggiori e con una dimensionalità più elevata permetterebbe di verificare se i pattern di degradazione identificati siano specifici per questo caso di studio clinico o se rappresentino proprietà universali dei sistemi di ML.